



CI/CD Best Practices for Your DevOps Journey

Agenda

- Beyond DevOps: Software Delivery Management
- Summary of 10 CI/CD best practices
- “Double-Click” on key best practices

DevOps Elite Performers

46x

more frequent code deploys
i.e. multiple times per day vs. once a week or less

2555x

faster lead-time from commit to deploy
i.e. less than an hour vs. more than a week

2406x

faster time to recover from downtime
i.e. less than an hour vs. weeks

1/7th

as likely that changes will fail
i.e. fail 0-15% of the time vs. 46-60%

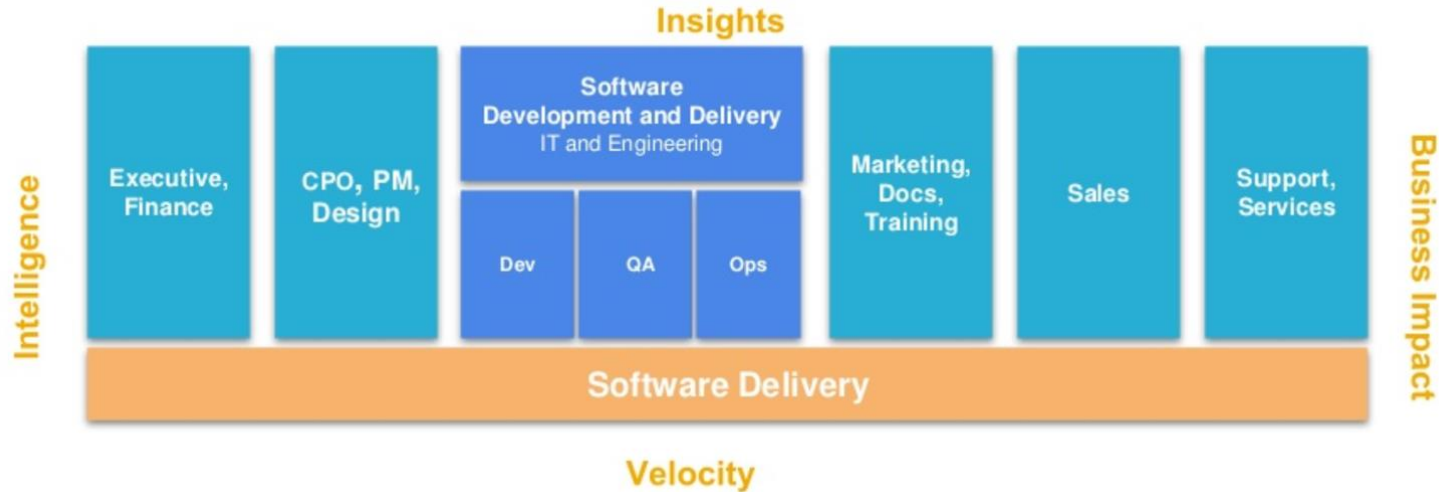
But... Things aren't perfect.

We still have...

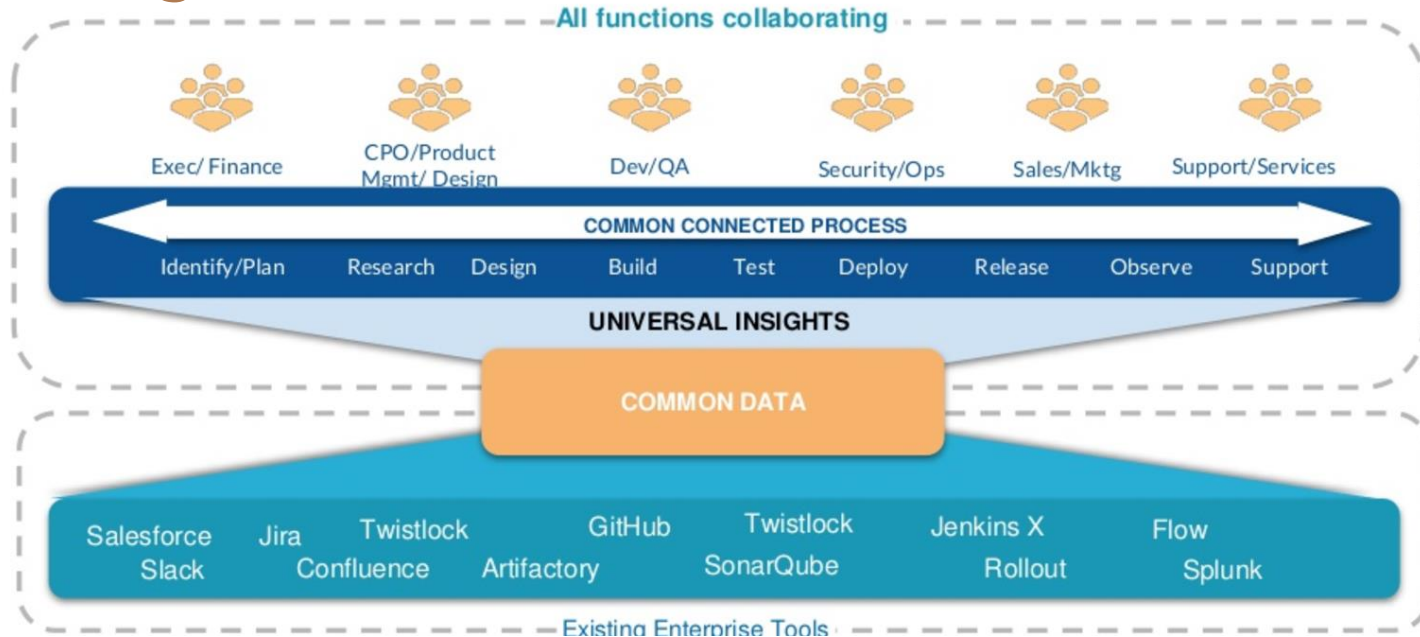
Disconnected and Fragmented teams, tools and process

- Disparate, stand-alone software tools
- No common language, data or process sets
- No clear way to ensure we deliver the right thing

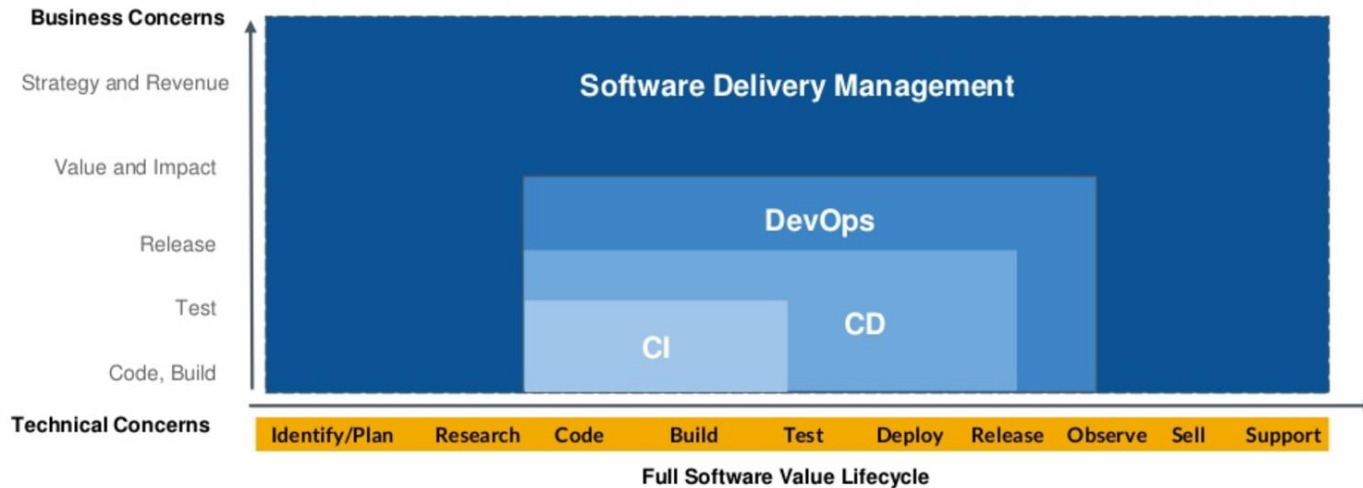
It Takes a Village to Truly Deliver Value



Introducing Software Delivery Management



We have to Start with CI/CD and DevOps



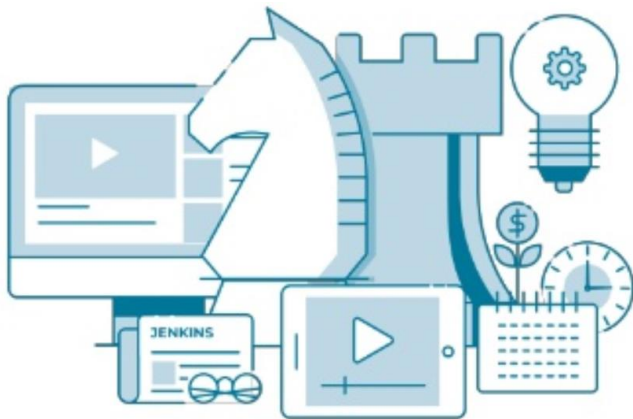
But the reality is...

Far fewer people are truly practicing CI/CD than is widely reported!



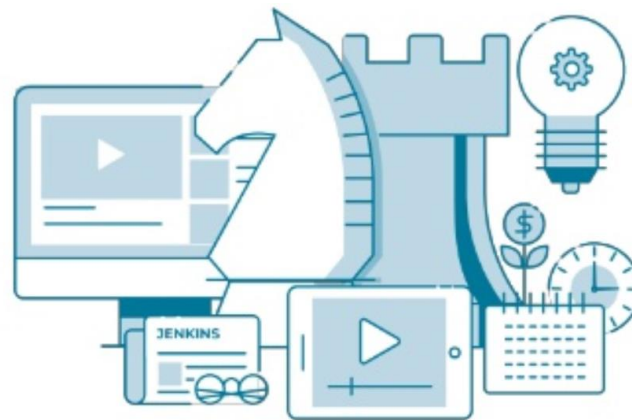
The Path to the Future

As the industry positions itself to build on DevOps practices with a Software Delivery Management strategy... it's more important than ever that we implement CI/CD best practices, and prepare for the future.



10 CI/CD Best Practices

1. **Track work items**
2. **Use source code management**
3. **Tags, not Branches**
4. **Automate the builds**
5. **Stop the line when the build breaks**
6. **Validate and test**
7. **Deploy**
8. **Improve incrementally**
9. **Collaboration**
10. **Create a true DevOps culture**

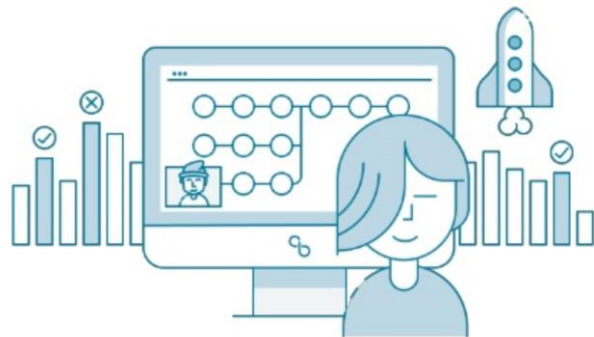


Best Practice #1: Track Work Items

Use a system that lets you track everything related to your release:

- User Stories
- Bugs
- Infrastructure and environment

This is useful for CI and CD, but is absolutely critical as you move to DevOps and eventually Software Delivery Management (SDM).

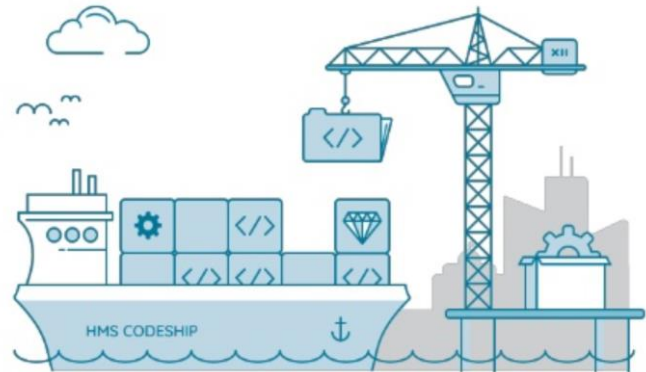


Best Practice #2: Use Source Code Management(SCM)

Use an SCM tool to track all the changes made for a release.

A decentralized version control system such as git A centralized version control system such as SVN

Must consider support for new and legacy systems



Use Source Code Management(SCM)

Encourage commit best practices

If culture is to not commit frequently, it won't matter. If a developer waits three weeks to commit or branches off for three weeks, they have delayed the integration and broken the principles

- Commit frequently
- Minimize duration of branches

Consider how triggers that launch builds are initiated.

- Trigger on every commit
- Trigger on developer branch
- Trigger on pull request
- Trigger on merge

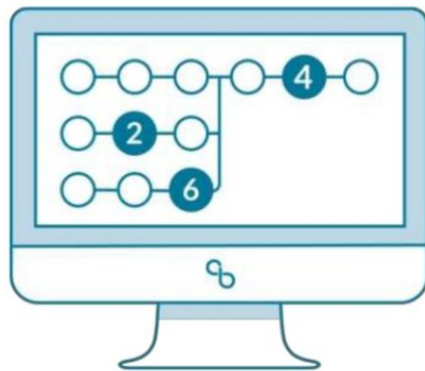
Best Practice #3: Tags, not Branches

Commit or Merge to the trunk frequently, or even better, always commit to the trunk/master

Developers often branch and maintain changes off of a trunk to manage releases.

But branching creates complexity that prevents everyone from working with a single source of truth, and introduces maintenance overhead

- Use tags to manage releases
- Implement functionality incrementally
- Use feature flags



Best Practice #4: Automate the Builds

Centralize and automate the entire build process

- Compiling the source code
- Packaging the compiled code
- Container images

The build process should run for every commit.

The build should run as quickly as possible The builds should take no more than 5-10 minutes. Commit should be blocked during build, to support “stop-the-line” culture. If builds are long developers defer commits.

- Build
- Code scans, unit test
- Functional tests if possible

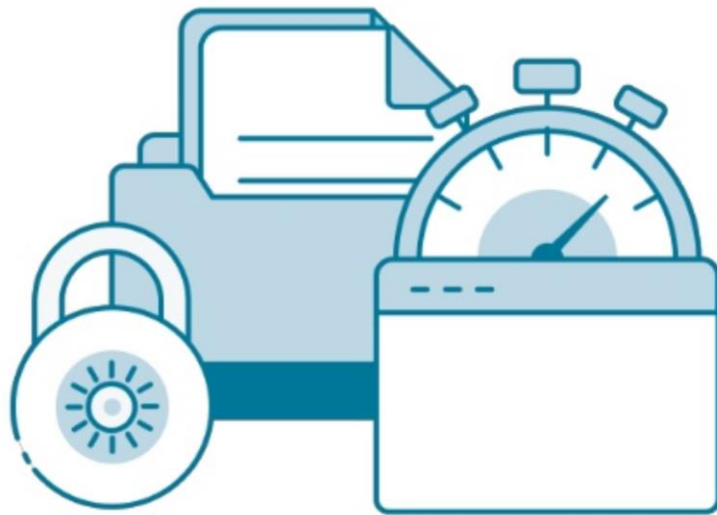
Promote the build to extended CI/CD loop for longer activities

Best Practice #6: Validation and Test

Organizations that don't validate every build are not practicing CI.

Organizations that do not continuously test are not practicing CD.

- Manage tests like code or as code
- Fail if the build if test thresholds aren't met
- Duplicate the production environment as much as possible



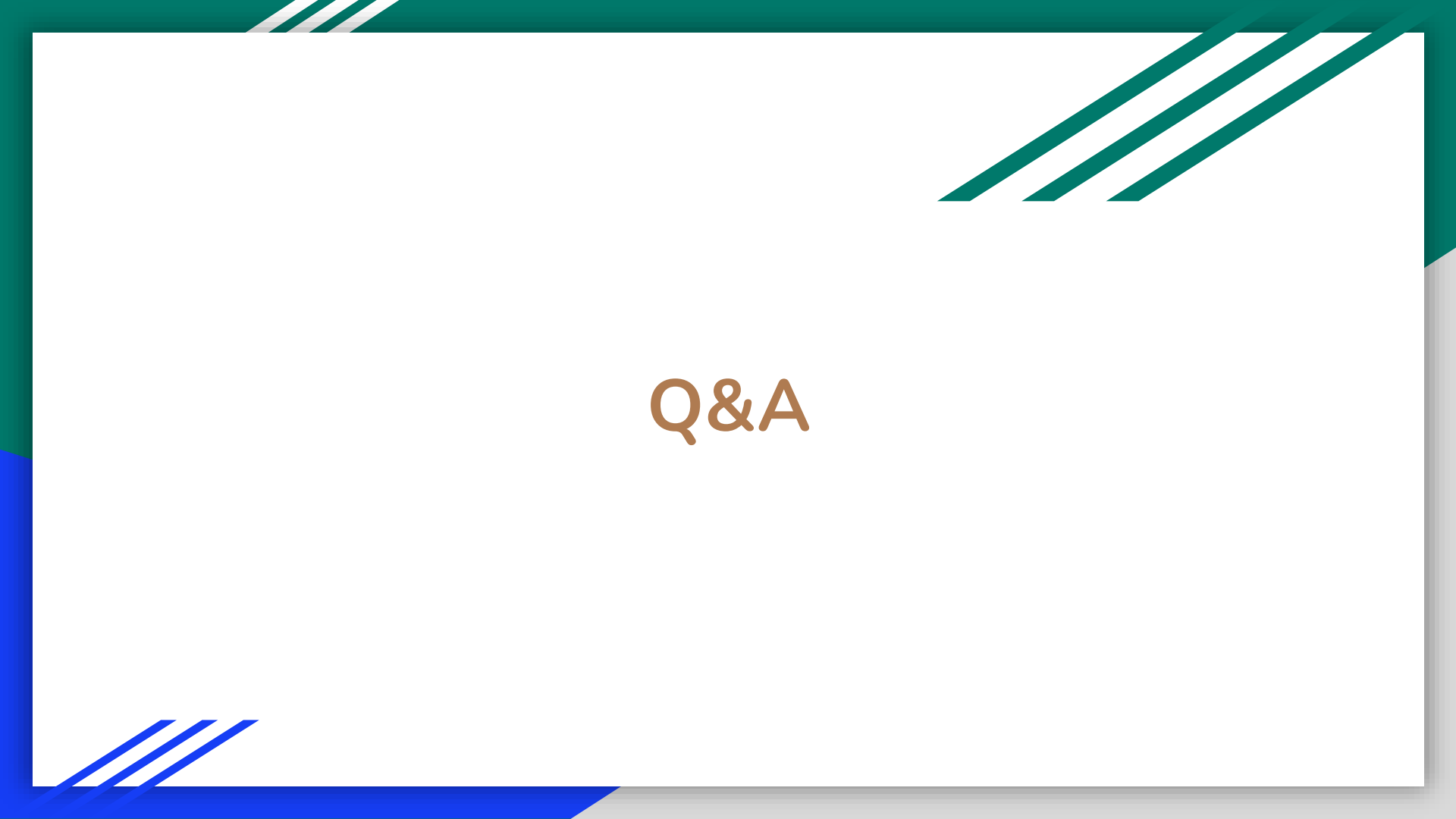
Best Practice #10: Create a True DevOps Culture

This may be the most difficult practice to implement, but no DevOps transformation is possible without it.

Starting a successful DevOps journey requires significant culture change, including:

- Executive support
- Training
- Funding
- A new mindset for everyone: the CIO/CTO, release managers, engineering managers, the ops team, and the dev team.

But..start, learn, and improve



Q&A